

MANUAL DE USUARIO

Sistema de Gestión Hospitalaria
Mateo Benjamin Ortiz de Leon
2024204

1. Introducción

Este manual describe el funcionamiento completo del Sistema de Gestión Hospitalaria, una API REST desarrollada con Node.js y TypeScript que permite administrar los recursos clínicos de un hospital: pacientes, médicos, especialidades, horarios, citas médicas, historial clínico, medicamentos, inventario y recetas.

La API no utiliza frameworks como Express; está construida sobre el módulo nativo http de Node.js y persiste todos los datos en una base de datos MySQL mediante stored procedures. Cada respuesta se entrega en formato JSON.

1.1 Objetivo del sistema

Proveer un servidor HTTP que exponga operaciones CRUD (Crear, Leer, Actualizar, Eliminar) para cada entidad del sistema hospitalario, validando la información recibida antes de persistirla y respondiendo con códigos HTTP estándar.

1.2 Tecnologías utilizadas

- Node.js con el módulo nativo http
- TypeScript — compilado y ejecutado con tsx
- MySQL como motor de base de datos
- mysql2 como driver de conexión
- pnpm como gestor de paquetes
- Arquitectura en capas: Routes → Controller → Service → Repository

2. Instalación y configuración

2.1 Requisitos previos

- Node.js v18 o superior
- pnpm instalado globalmente (npm install -g pnpm)
- Servidor MySQL activo
- Cliente de pruebas HTTP como Postman

2.2 Configuración de la base de datos

Antes de iniciar el servidor es necesario crear la base de datos. Abrir el cliente MySQL (Workbench, DBeaver o línea de comandos) y ejecutar el archivo SQL proporcionado:

```
BDgestionHospitalaria_in5cm.sql
```

Este script crea la base de datos DBgestionHospitalaria_in5cm, todas las tablas, los stored procedures y un conjunto de registros iniciales de prueba que incluyen especialidades, médicos, pacientes, citas, historial clínico, medicamentos e inventario.

2.3 Variables de entorno

En la raíz del proyecto existe el archivo .env con la configuración de conexión a la base de datos. Editarlo con los datos del servidor MySQL local:

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=IN5CM
DB_PASSWORD=?donmoA5m@
DB_NAME=DBgestionHospitalaria_in5cm
PORT=3000
```

2.4 Instalación de dependencias

Desde la carpeta raíz del proyecto ejecutar:

```
pnpm install
```

2.5 Ejecución del servidor

```
pnpm run dev
```

El servidor queda disponible en <http://localhost:3000>. Para confirmar que está corriendo correctamente se puede hacer una petición GET a cualquier ruta de listado, por ejemplo:

```
GET http://localhost:3000/api/pacientes
```

3. Estructura del proyecto

El proyecto sigue una arquitectura en capas donde cada capa tiene una responsabilidad única:

3.1 Capas de la arquitectura

Routes: Reciben la petición HTTP, identifican el método y el ID si aplica, y delegan al controlador correspondiente.

Controllers: Leen el cuerpo de la petición (body), llaman al servicio y envían la respuesta JSON al cliente.

Services: Contienen la lógica de negocio y las validaciones. Si faltan campos requeridos, lanzan un error antes de llegar al repositorio.

Repositories: Son la única capa que habla con MySQL. Cada método ejecuta un stored procedure específico.

3.2 Flujo de una petición

Cuando llega una petición al servidor, el router central (src/routes/router.ts) la recibe y determina el recurso a partir del segundo segmento de la URL (/api/recurso). Luego invoca el archivo de rutas correspondiente, que identifica el método HTTP y el ID para decidir qué método del controlador ejecutar. El controlador lee el body, llama al servicio, y el servicio llama al repositorio que ejecuta el stored procedure en MySQL.

3.3 Manejo de errores

El servidor responde con códigos HTTP estándar según el caso:

- 200 OK — operación exitosa (GET, PUT, PATCH, DELETE)
- 201 Created — recurso creado exitosamente (POST)
- 400 Bad Request — campos faltantes o datos inválidos
- 404 Not Found — ruta o recurso inexistente
- 405 Method Not Allowed — método HTTP no soportado en esa ruta
- 500 Internal Server Error — error inesperado en el servidor

4. Módulo de Login

El módulo de login valida las credenciales de un usuario registrado en la tabla Login de la base de datos. No genera tokens; simplemente verifica que el correo y la contraseña coincidan con un registro activo y retorna los datos del usuario.

4.1 Endpoint

Método: POST

URL: `http://localhost:3000/api/login`

4.2 Cuerpo de la petición

```
{
  "correo": "admin1@gmail.com",
  "contrasena": "12345"
}
```

4.3 Usuarios de prueba disponibles

- `admin1@gmail.com` — rol Administrador
- `medico1@gmail.com` — rol Medico
- `medico2@gmail.com` — rol Medico
- `enfermero1@gmail.com` — rol Enfermero

Todos los usuarios de prueba tienen la contraseña 12345.

4.4 Respuesta exitosa

Retorna el id, correo, nombre de usuario, rol y estado del login encontrado.

4.5 Errores posibles

- 400 — "Correo y contrasena son requeridos" si alguno de los dos campos viene vacío
- 400 — "Credenciales invalidas" si el correo o la contraseña no coinciden con ningún registro activo

5. Módulo de Especialidades

Administra las especialidades médicas disponibles en el hospital. Cada especialidad tiene un nombre y una descripción. El estado (activo/inactivo) lo gestiona el sistema internamente.

5.1 Listar especialidades

Método: GET

URL: `http://localhost:3000/api/especialidades`

Retorna todas las especialidades con estado activo, ordenadas alfabéticamente por nombre.

5.2 Agregar especialidad

Método: POST

URL: `http://localhost:3000/api/especialidades`

```
{
  "nombre": "Ginecología",
  "descripcion": "Salud del sistema reproductivo femenino"
}
```

El campo nombre es obligatorio. El campo descripcion es opcional; si se omite se guarda como cadena vacía.

5.3 Actualizar especialidad

Método: PUT

URL: `http://localhost:3000/api/especialidades/:id`

Reemplazar :id por el número de la especialidad a actualizar. El body es idéntico al del POST.

5.4 Eliminar especialidad

Método: DELETE

URL: `http://localhost:3000/api/especialidades/:id`

El sistema realiza un eliminado lógico: cambia el estado interno a inactivo, el registro no desaparece de la base de datos.

6. Módulo de Pacientes

Gestiona el registro completo de pacientes. Cada paciente tiene datos personales, de contacto e información médica básica como tipo de sangre y alergias.

6.1 Listar pacientes

Método: GET

URL: `http://localhost:3000/api/pacientes`

Retorna todos los pacientes activos junto con el usuario de login asociado si existe.

6.2 Buscar paciente por ID

Método: GET

URL: `http://localhost:3000/api/pacientes/:id`

6.3 Agregar paciente

Método: POST

URL: `http://localhost:3000/api/pacientes`

```
{
  "nombre": "Carlos",
  "apellido": "López",
  "fechaNacimiento": "1995-03-20",
  "genero": "Masculino",
  "telefono": "55551234",
  "direccion": "Zona 7, Guatemala",
  "tipoSangre": "O+",
  "alergias": "Ninguna"
}
```

Campos obligatorios: nombre, apellido, fechaNacimiento, genero, telefono. Los campos direccion, tipoSangre y alergias son opcionales.

6.4 Actualizar paciente

Método: PUT

URL: `http://localhost:3000/api/pacientes/:id`

El body es idéntico al del POST con los mismos campos obligatorios.

6.5 Eliminar paciente

Método: DELETE

URL: `http://localhost:3000/api/pacientes/:id`

Eliminado lógico: cambia el estado del paciente a inactivo.

7. Módulo de Médicos

Gestiona el registro de los médicos del hospital. Cada médico está vinculado a una especialidad y puede tener un usuario de login asociado.

7.1 Listar médicos

Método: GET

URL: `http://localhost:3000/api/medicos`

Retorna todos los médicos activos con el nombre de su especialidad y su usuario de login.

7.2 Agregar médico

Método: POST

URL: `http://localhost:3000/api/medicos`

```
{
  "idLogin": 2,
  "idEspecialidad": 1,
  "nombre": "Ricardo",
  "apellido": "Pérez",
  "telefono": "55530001",
  "numeroColegiado": "MED-007"
}
```

El campo `idLogin` hace referencia a un registro de la tabla Login. El campo `idEspecialidad` debe corresponder a una especialidad existente.

7.3 Actualizar médico

Método: PUT

URL: `http://localhost:3000/api/medicos/:id`

```
{
  "idEspecialidad": 2,
  "nombre": "Ricardo",
  "apellido": "Pérez",
  "telefono": "55530001",
  "numeroColegiado": "MED-007"
}
```

La actualización no modifica el `idLogin`, solo los datos clínicos y de contacto.

7.4 Eliminar médico

Método: DELETE

URL: `http://localhost:3000/api/medicos/:id`

Eliminado lógico: el médico pasa a estado inactivo.

8. Módulo de Horarios

Permite registrar los horarios de atención de cada médico por día de la semana. Un médico puede tener múltiples horarios en diferentes días.

8.1 Listar horarios

Método: GET

URL: `http://localhost:3000/api/horarios`

Retorna todos los horarios activos con el nombre del médico asociado, ordenados por día y hora de inicio.

8.2 Horarios por médico

Método: GET

URL: `http://localhost:3000/api/horarios/medico/:idMedico`

Filtra y retorna únicamente los horarios activos del médico indicado.

8.3 Agregar horario

Método: POST

URL: `http://localhost:3000/api/horarios`

```
{
  "idMedico": 1,
  "diaSemana": "Lunes",
  "horaInicio": "08:00",
  "horaFin": "14:00"
}
```

Los valores válidos para `diaSemana` son: Lunes, Martes, Miercoles, Jueves, Viernes, Sabado, Domingo.

8.4 Actualizar horario

Método: PUT

URL: `http://localhost:3000/api/horarios/:id`

```
{
  "diaSemana": "Martes",
  "horaInicio": "09:00",
  "horaFin": "13:00"
}
```

La actualización no modifica el médico asociado al horario.

8.5 Eliminar horario

Método: DELETE

URL: `http://localhost:3000/api/horarios/:id`

Eliminado lógico.

9. Módulo de Citas

Gestiona las citas médicas del hospital. Una cita vincula a un paciente con un médico en una fecha y hora determinadas. El estado de la cita puede actualizarse de forma independiente mediante PATCH.

9.1 Listar citas

Método: GET

URL: `http://localhost:3000/api/citas`

Retorna todas las citas con el nombre del paciente y del médico, ordenadas por fecha y hora descendente.

9.2 Citas por paciente

Método: GET

URL: `http://localhost:3000/api/citas/paciente/:idPaciente`

Retorna el historial de citas de un paciente específico junto con el nombre del médico.

9.3 Agregar cita

Método: POST

URL: `http://localhost:3000/api/citas`

```
{
  "idPaciente": 1,
  "idMedico": 1,
  "fecha": "2026-08-15",
  "hora": "09:00",
  "motivo": "Revisión general"
}
```

El estado inicial se asigna automáticamente como Pendiente. No es necesario enviarlo en el body.

9.4 Actualizar cita

Método: PUT

URL: `http://localhost:3000/api/citas/:id`

```
{
  "fecha": "2026-08-20",
  "hora": "10:00",
  "motivo": "Revisión general actualizada",
  "estado": "Confirmada"
}
```

9.5 Cambiar estado de la cita

Método: PATCH

URL: `http://localhost:3000/api/citas/:id/estado`

```
{  
  "estado": "Completada"  
}
```

Estados válidos: Pendiente, Confirmada, Completada, Cancelada.

9.6 Eliminar cita

Método: DELETE

URL: `http://localhost:3000/api/citas/:id`

A diferencia de otras entidades, las citas se eliminan físicamente de la base de datos (DELETE real, no lógico).

10. Módulo de Historial Clínico

Registra las visitas médicas de los pacientes. Cada entrada del historial documenta el diagnóstico, tratamiento, medicamentos indicados y observaciones del médico. La fecha de visita la genera automáticamente la base de datos al momento de insertar.

10.1 Listar historiales

Método: GET

URL: `http://localhost:3000/api/historiales`

Retorna todos los historiales clínicos con el nombre del paciente y del médico, ordenados por fecha descendente.

10.2 Historial por paciente

Método: GET

URL: `http://localhost:3000/api/historiales/paciente/:idPaciente`

Retorna todas las entradas del historial clínico de un paciente específico.

10.3 Agregar historial

Método: POST

URL: `http://localhost:3000/api/historiales`

```
{
  "idPaciente": 1,
  "idMedico": 1,
  "diagnostico": "Hipertensión leve",
  "tratamiento": "Reposo y dieta baja en sodio",
  "medicamentos": "Losartán 50mg",
  "observaciones": "Revisión en 30 días"
}
```

No se envía fechaVisita porque el stored procedure la registra automáticamente con la fecha y hora actuales del servidor de base de datos.

El módulo de historial clínico no cuenta con endpoints de actualización ni eliminación; los registros son permanentes una vez creados.

11. Módulo de Medicamentos

Gestiona el catálogo de medicamentos del hospital. Cada medicamento tiene su nombre comercial, forma de presentación, concentración, laboratorio fabricante y una indicación de si requiere receta médica para su dispensación.

11.1 Listar medicamentos

Método: GET

URL: `http://localhost:3000/api/medicamentos`

Retorna todos los medicamentos activos, ordenados alfabéticamente por nombre.

11.2 Agregar medicamento

Método: POST

URL: `http://localhost:3000/api/medicamentos`

```
{
  "nombre": "Ibuprofeno",
  "presentacion": "Tableta",
  "concentracion": "400mg",
  "laboratorio": "FarmaGT",
  "requiereReceta": false
}
```

El campo `requiereReceta` acepta `true` o `false`. El estado del medicamento es activo por defecto.

11.3 Actualizar medicamento

Método: PUT

URL: `http://localhost:3000/api/medicamentos/:id`

El body es idéntico al del POST.

11.4 Eliminar medicamento

Método: DELETE

URL: `http://localhost:3000/api/medicamentos/:id`

Eliminado lógico: el medicamento pasa a estado inactivo.

12. Módulo de Inventario

Controla el stock de medicamentos disponibles en la farmacia del hospital. Cada entrada de inventario corresponde a un medicamento con una cantidad disponible, una cantidad mínima de alerta y una fecha de vencimiento. El módulo incluye una ruta especial para detectar medicamentos con stock bajo.

12.1 Listar inventario

Método: GET

URL: `http://localhost:3000/api/inventario`

Retorna todos los registros de inventario activos junto con el nombre del medicamento, ordenados por fecha de vencimiento ascendente.

12.2 Stock bajo

Método: GET

URL: `http://localhost:3000/api/inventario/stock-bajo`

Retorna los registros cuya cantidad disponible es menor o igual a la cantidad mínima definida. Útil para generar alertas de reabastecimiento.

12.3 Agregar al inventario

Método: POST

URL: `http://localhost:3000/api/inventario`

```
{
  "idMedicamento": 1,
  "cantidad": 100,
  "cantidadMinima": 10,
  "fechaVencimiento": "2027-12-31"
}
```

El campo `cantidad` representa las unidades disponibles al momento del registro. El campo `cantidadMinima` define el umbral de alerta para stock bajo.

12.4 Actualizar inventario

Método: PUT

URL: `http://localhost:3000/api/inventario/:id`

El body es idéntico al del POST. Permite corregir la cantidad mínima o la fecha de vencimiento.

12.5 Ajustar stock

Método: PATCH

URL: `http://localhost:3000/api/inventario/:id/stock`

```
{
  "cantidad": 50
}
```

```
}
```

Este endpoint suma la cantidad indicada al stock actual. Por ejemplo, si hay 30 unidades y se envía 50, el resultado será 80. Para descontar stock se puede enviar un número negativo.

12.6 Eliminar del inventario

Método: DELETE

URL: <http://localhost:3000/api/inventario/:id>

Eliminado lógico.

13. Módulo de Recetas

Registra las recetas médicas emitidas por los médicos. Cada receta vincula una entrada del historial clínico con un medicamento del inventario, especificando la dosis, duración y cantidad recetada. El estado de la receta puede actualizarse de Pendiente a Dispensada una vez que la farmacia entrega el medicamento.

13.1 Listar recetas

Método: GET

URL: `http://localhost:3000/api/recetas`

Retorna todas las recetas con el nombre del paciente, médico y medicamento asociados, ordenadas por fecha descendente.

13.2 Agregar receta

Método: POST

URL: `http://localhost:3000/api/recetas`

```
{
  "idHistorial": 1,
  "idMedico": 1,
  "idPaciente": 1,
  "idInventario": 1,
  "dosis": "1 tableta cada 8 horas",
  "duracionDias": 7,
  "cantidad": 21
}
```

La fecha de la receta la genera automáticamente la base de datos. El estado inicial es Pendiente.

13.3 Cambiar estado de la receta

Método: PATCH

URL: `http://localhost:3000/api/recetas/:id/estado`

```
{
  "estado": "Dispensada"
}
```

Estados válidos: Pendiente, Dispensada. Se utiliza cuando la farmacia entrega el medicamento al paciente.

14. Referencia rápida de endpoints

Base URL: `http://localhost:3000`

Login

```
POST    /api/login
```

Especialidades

```
GET      /api/especialidades
POST     /api/especialidades
PUT      /api/especialidades/:id
DELETE   /api/especialidades/:id
```

Pacientes

```
GET      /api/pacientes
GET      /api/pacientes/:id
POST     /api/pacientes
PUT      /api/pacientes/:id
DELETE   /api/pacientes/:id
```

Médicos

```
GET      /api/medicos
POST     /api/medicos
PUT      /api/medicos/:id
DELETE   /api/medicos/:id
```

Horarios

```
GET      /api/horarios
GET      /api/horarios/medico/:idMedico
POST     /api/horarios
PUT      /api/horarios/:id
DELETE   /api/horarios/:id
```

Citas

```
GET      /api/citas
```

```
GET      /api/citas/paciente/:idPaciente
POST     /api/citas
PUT      /api/citas/:id
PATCH   /api/citas/:id/estado
DELETE   /api/citas/:id
```

Historial Clínico

```
GET      /api/historiales
GET      /api/historiales/paciente/:idPaciente
POST     /api/historiales
```

Medicamentos

```
GET      /api/medicamentos
POST     /api/medicamentos
PUT      /api/medicamentos/:id
DELETE   /api/medicamentos/:id
```

Inventario

```
GET      /api/inventario
GET      /api/inventario/stock-bajo
POST     /api/inventario
PUT      /api/inventario/:id
PATCH   /api/inventario/:id/stock
DELETE   /api/inventario/:id
```

Recetas

```
GET      /api/recetas
POST     /api/recetas
PATCH   /api/recetas/:id/estado
```